

F.Y. M.C.A. (Management)
STQ-553 MJ : SOFTWARE TESTING AND QUALITY ASSURANCE
(2024 Pattern) (Semester - II)

Question Paper Solution Nov /Dec 2025

Time : 2½ Hours]

[Max. Marks : 50

Instructions to the candidates:

- 1) All questions are compulsory.
- 2) All questions carry equal marks.
- 3) Draw diagram whenever required

Question 1(5 Marks Each) (Chapter 1)

A. Describe Software Quality Assurance & Its Challenges

Answer

Definition:

SQA is a systematic process of evaluating the quality of software during its development lifecycle. It ensures that the software meets the specified requirements and standards, reducing defects and improving the overall reliability of the product.

Objective:

- Make sure the work is done as per the rules and guidelines.
- Find and fix issues as soon as possible during development.
- Ensure the software works well, lasts long, and performs reliably.
- Deliver a product that meets customer expectations and earns their confidence.

Software Quality Assurance (SQA) Challenges:

- **Changing Requirements:** Frequent changes in requirements can disrupt planned processes and make it harder to ensure quality.
- **Tight Deadlines:** Limited time for development and testing can lead to incomplete testing or skipping quality checks.

- **Resource Constraints:** Insufficient resources, such as skilled personnel, tools, or budget, can affect the quality assurance process.
- **Integration Issues:** Ensuring seamless integration of different modules and components can be complex, especially in large systems.
- **Communication Gaps:** Miscommunication between teams, such as developers, testers, and clients, can lead to misunderstandings about requirements or priorities.
- **Complex Systems:** Testing complex systems with numerous dependencies and interactions can be challenging and time-consuming.
- **Lack of Proper Tools:** Without the right testing tools or automation, the QA process can become inefficient and error-prone.
- **Emerging Technologies:** Rapid advancements in technology require QA teams to stay updated, which can be difficult.
- **Ensuring Security:** Identifying and addressing vulnerabilities to ensure data security and compliance is a major challenge in modern software.
- **Cross-Platform Compatibility:** Ensuring the software works across various devices, operating systems, and browsers adds complexity to the QA process.

B) Illustrate ISO 9000 standards in SQA

Answer

The International Organization for Standardization 9000 standards are a set of international quality management standards used to ensure that organizations maintain high-quality products and services. In Software Quality Assurance (SQA), ISO 9000 helps software companies improve development processes, maintain consistency, and satisfy customer requirements.

Main Features of ISO 9000 in SQA

1. **Quality Management System (QMS):** ISO 9000 provides guidelines to create a proper quality management system for software development.
2. **Customer Focus:** The standards ensure that customer requirements are understood and fulfilled properly.
3. **Process-Oriented Approach:** It focuses on improving software development processes rather than only checking the final product.

4. **Documentation and Standards:** Proper documentation of coding, testing, maintenance, and project management is maintained.
5. **Continuous Improvement:** Organizations regularly improve their software processes and quality practices.
6. **Error Prevention:** ISO 9000 emphasizes preventing defects before they occur instead of correcting them later.

Important ISO 9000 Standards

- **ISO 9001:** This is the most widely used standard for quality management systems. It defines requirements for maintaining software quality and customer satisfaction.
- **ISO 9000:** It explains the basic concepts, principles, and vocabulary related to quality management systems.
- **ISO 9004:** It provides guidelines for improving organizational performance and achieving long-term success.

Benefits of ISO 9000 in SQA

- Improves software quality
- Increases customer satisfaction
- Reduces software defects
- Enhances process efficiency
- Improves documentation and management
- Builds customer trust and company reputation

C. Distinguish between software quality assurance and software quality control.

Answer

Quality Assurance /QA	Quality Control / QC
QA focuses on providing assurance that the quality requested will be achieved	QC focuses on fulfilling the quality requested
QA is the technique of managing quality	QC is the technique to verify quality
QA focuses on preventing defect by improving & optimizing the processes used to develop software	QC focuses on identifying & fixing defect in the final product
QA is process oriented	QC is product oriented
QA prevent & minimizes software defects during development time	QC identifies & fixes the defects that exists after development
QA is a managerial tool	QC is corrective tool
QA is performed before QC	QC is performed after the QA activity is done
QA is preventive technique	QC is corrective technique
QA is low level activity that identify an error & mistake	QC is high level activity that identify an error that QA cannot
QA is less time consuming	QC is more time consuming process
Eg.Verification	Eg.Validation

D. Define Reliability and explain the measures Availability, ROCOF, POFOD

Answer

Reliability is the ability of a software system or product to perform its required functions correctly without failure for a specified period of time under given conditions. It shows how dependable and stable the software is during operation.

Example: If an online banking system works continuously without crashing or giving wrong results, it is considered reliable.

1) Availability: Availability measures how much time the system is operational and accessible when required.

Formula

$$Availability = \frac{MTBF}{MTBF + MTTR}$$

Where:

- MTBF (Mean Time Between Failures) = Average working time before failure occurs.
- MTTR (Mean Time To Repair) = Average time required to repair the system after failure.

This formula shows the proportion of time the system remains operational.

Example: If MTBF = 90 hours and MTTR = 10 hours,

$$Availability = \frac{90}{90 + 10} = 0.9$$

So, Availability = 90%.

2) Rate of Occurrence of Failures (ROCOF)

ROCOF measures the frequency of software failures over a specified period of operating time. It shows how often the system fails during use.

Formula

$$ROCOF = \frac{\text{Number of Failures}}{\text{Total Operating Time}}$$

Example : If an application crashes 5 times in 10 hours of operation:

$$ROCOF = \frac{5}{10} = 0.5$$

So, the ROCOF is **0.5 failures per hour**.

This means the application fails once every 2 hours. Lower ROCOF indicates a more reliable system.

3) Probability of Failure on Demand (POFOD)

POFOD measures the probability that a system will fail when a specific service or request is made.

It shows how likely the system is to fail on demand.

Formula

$$POFOD = \frac{\text{Number of Failed Requests}}{\text{Total Number of Requests}}$$

Example: If a website handles 1000 requests and fails for 2 requests:

$$POFOD = \frac{2}{1000} = 0.002$$

So, the POFOD is **0.002 or 0.2%**.

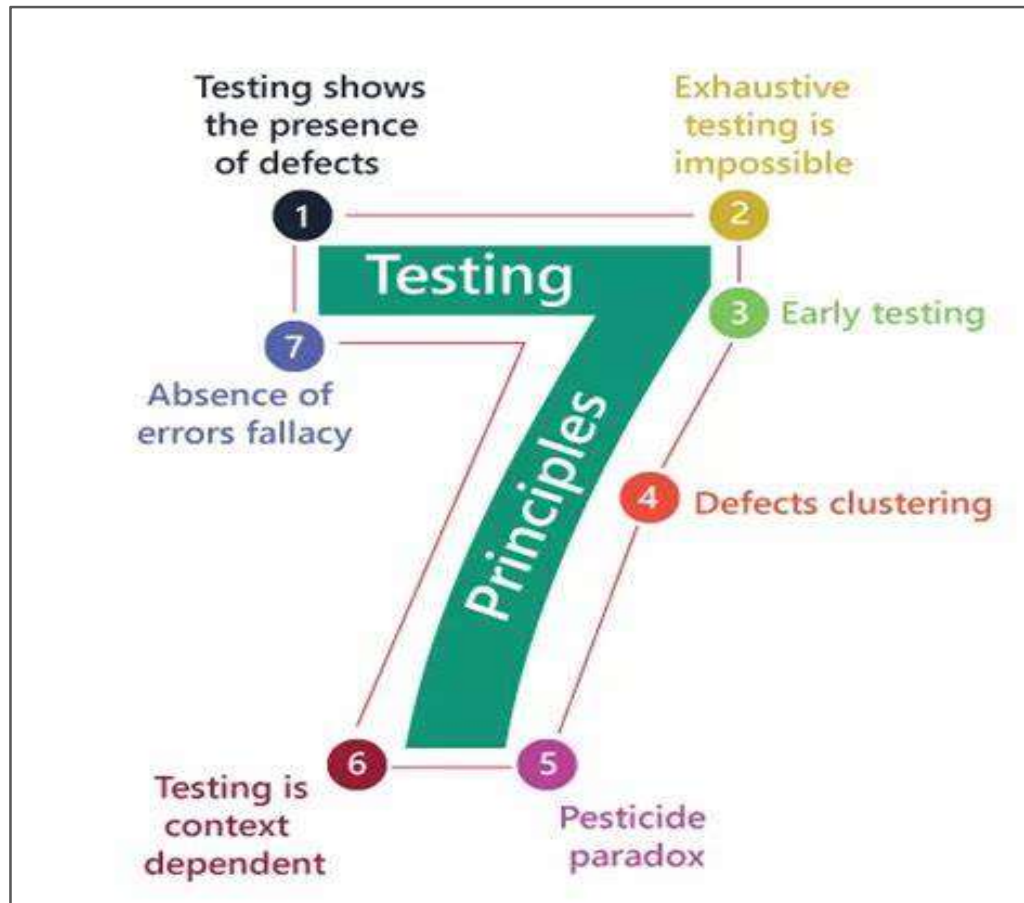
This means the system fails for 2 requests out of every 1000 requests. Lower POFOD indicates a more reliable system.

Question 2 (5 Marks Each)

A. Explain the Testing Principles.

Answer

There are total 7 testing Principles



1. Testing shows the presence of defects

Testing can't prove that the software is completely bug-free. It can only show that there *are* defects. So, the goal of testing is to find problems, not to guarantee perfection.

Example: Just because a test passes, doesn't mean there are no hidden bugs.

2. Exhaustive testing is impossible

Exhaustive testing is impossible because software systems have too many possible inputs, conditions, and user actions. Testing every possible test case requires huge time, cost, and resources. Therefore, testers cannot test software in every situation. Instead, they focus on important and risky parts of the software to find maximum defects.

Example, an ATM system has thousands of combinations of PINs, transactions, and user actions, so testing every combination is not practical. Hence, exhaustive testing is practically impossible in software testing.

Example: It's impossible to test every combination of buttons a user might press, so you focus on the main features.

3. Early testing

Testing should start as early as possible in the development process. Finding and fixing bugs early is cheaper and easier than finding them later.

Example: Testing the first version of the code helps catch bugs before the program grows more complex.

4. Defects cluster together / Defect Clustering

This principle means that if one part of the software has a bug, there are likely to be more bugs in the same area. It's like if one part of a machine breaks down, other parts nearby might also be broken or need fixing.

Example: If there's a bug in the login feature, you might also find other bugs in related areas like the password reset or user authentication. So, instead of testing random parts of the software, testers focus on the parts that are most likely to have multiple issues.

5. Beware of the pesticide paradox

Running the same tests over and over won't help find new bugs. You need to change your test cases or create new ones to find different defects.

Example: If you always run the same test for a feature, you may not discover new bugs. You need to try different test scenarios.

6. Testing is context-dependent

The way you test software depends on the type of software, its purpose, and how critical it is. Testing for a mobile app is different from testing for a banking system.

Example: A simple mobile app may only need basic testing, but a banking app needs thorough security testing to protect users' data.

7. Absence of errors is a fallacy

If a built software is 99% bug-free but does not follow the user requirement then it is unusable. It is

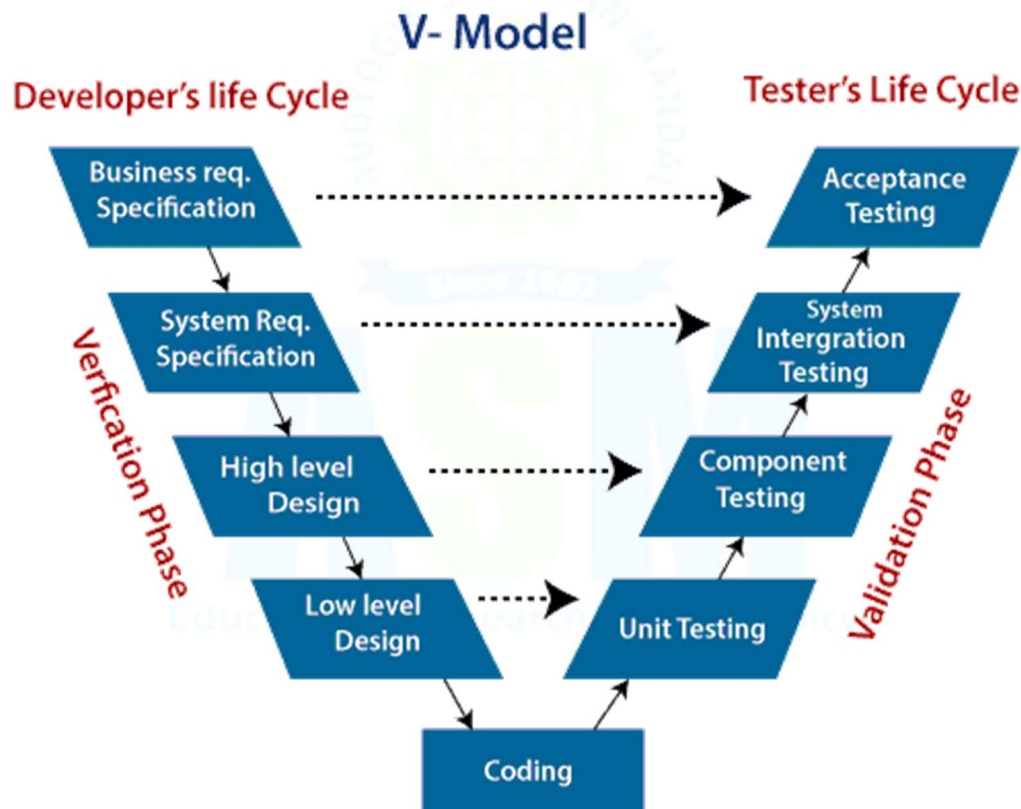
not only necessary that software is 99% bug-free but it is also mandatory to fulfill all the customer requirements.

Example: The app may run perfectly without crashes, but if it doesn't give users the features they expect, it's still a failure.

B. Illustrate V model phases

Answer

V-Shaped Structure: The process is visually represented in the shape of the letter "V". The left side of the "V" focuses on development activities (such as requirement analysis, design, coding), and the right side represents testing activities (such as unit testing, integration testing, and system testing).



Verification: It ensures that the product is being built **correctly** (i.e., it meets the specified requirements). This process includes reviews, walkthroughs, and inspections.

1. Business Requirement Specification

- The project team gathers business requirements from stakeholders.
- These requirements define what the system should do.
- **Corresponding Testing: Acceptance Testing** (Ensures the final product meets

business needs).

2. System Requirement Specification

- Detailed system requirements are defined based on business needs.
- Includes functional and non-functional requirements.
- **Corresponding Testing: System Integration Testing** (Verifies that system components work together).

3. High-Level Design (HLD)/Architectural Design

- Defines system architecture, modules, and overall structure.
- Focuses on **how** the system will function.
- **Corresponding Testing: Component Testing** (Ensures that modules are correctly integrated).

4. Low-Level Design (LLD)/Module Design

- Detailed design of individual components, classes, and functions.
- Defines data structures, algorithms, and internal logic.
- **Corresponding Testing: Unit Testing** (Verifies individual components work as expected).

5. Coding

- The actual implementation of software modules takes place based on LLD.
- The **output** of this phase is the source code of the application.
- Next, the validation phase starts to test the code.

Validation (Testing Side): It ensures that the **right product** is being built (i.e., it meets the user's needs and expectations). This includes testing and user acceptance activities.

1. Unit Testing (Low-Level Testing)

- Tests individual units (functions, methods, classes).
- Performed using tools like JUnit, NUnit, and PyTest.

2. Component Testing (Integration Testing)

- Tests how different components interact with each other.
- Ensures that data flow between modules works correctly.

3. System Integration Testing (SIT)

- Verifies that the entire system works as a whole.
- Focuses on interfaces, APIs, and external system interactions.

4. Acceptance Testing

- Ensures that the final system meets business requirements.

- Usually performed by the **end-users** before deployment.

C. Discuss any five non-functional testing types.

Answer

2.11.1. Performance (Load & Stress)

- **Performance Testing** checks how fast the software works under normal conditions.

Example: Testing if an e-commerce website loads quickly when 100 people are shopping at the same time.

- **Load Testing** checks how the software performs when many users are using it at once.

Example: Seeing if a ticket booking app works smoothly when many people try to buy concert tickets at the same time.

- **Stress Testing** checks how the software behaves under extreme conditions (more than usual usage).

Example: Adding thousands of users to an app at once to see when it crashes and if it can recover after crashing.

2. Usability: Focuses on the user interface and overall user experience.

Ensures that the application is easy to use and provides a satisfactory experience to end-users.

Example: Testing a mobile banking app to see if users can easily transfer money or check their balance without confusion.

3. Maintainability: Evaluates how easily the system can be maintained, including fixing bugs, updating code, and making modifications.

Example: Making sure the code of a shopping website is clean and well-documented so developers can quickly add new features or fix bugs.

4. Portability: Checks whether the software can operate on different hardware platforms, operating systems, browsers, or devices. Ensures that the software is adaptable and works seamlessly across various environments without requiring significant changes.

Example: Testing a game to see if it runs smoothly on Windows, Mac, Android, and iOS without any issues.

5. Security : Focuses on identifying vulnerabilities, risks, and threats in the software application. Ensures data integrity, confidentiality, and protection against unauthorized access, hacking, and other security breaches.

Example: Testing an online banking app to make sure that only the account holder can access their account and transactions are secure.

D. Differentiate between Re-testing and Regression Testing.

Answer

Re-testing	Regression Testing
Testing the specific part of the software where a defect was fixed.	Testing the entire software to ensure that recent changes haven't broken existing functionality.
Focuses only on the specific bug or issue that was fixed.	Focuses on checking the overall system after any new code changes.
To confirm that a specific bug has been fixed correctly.	To ensure that new changes (fixes, updates) don't negatively affect the existing system.
Performed after a defect is fixed.	Performed after any code change, such as bug fixes, new features, or updates.
Narrow scope (tests only the fixed part).	Broader scope (tests many or all parts of the system).
If a login bug is fixed, only the login function is tested again.	After the login bug is fixed, the entire app is tested to ensure other features (like sign-up, profile, etc.) still work.
Re-testing is a planned test.	Regression testing is generic testing.
We do retesting only on failed test cases.	We do regression testing on passed test cases
Priority of retesting is higher than regression testing	Priority of regression testing is lower than retesting
Retesting is carried out before Regression testing.	Regression testing is carried out in parallel with retesting
We cannot automate the test cases for Retesting due to uncertainty	We can automate regression test cases

Question 3 (5 Marks Each)

A . You are asked to test an Age Verification system that accepts the age of a person. The system allows users who are between 18 and 60 years old to access certain features. Using the Boundary value Analysis technique.

- i) Determine the number of test cases required to test the above functionality.
- ii) Specify the test cases, including representative values and their expected outcomes.

Answer

Given:

Minimum Age: 18

Maximum Age: 60

Valid Range :18 to 60

i) Number of test case using boundary value analysis:

1. Minimum Age -1
2. Minimum Age
3. Minimum Age + 1
4. Maximum Age -1
5. Maximum Age
6. Maximum + 1

So, **Total 6** Test Cases required to test functionality effectively

ii) Test Cases with Expected Outcome

Test Case Id	Input Age	Boundary Condition	Expected Outcome
TC 1	17	Minimum Age -1	Not Allowed/Invalid
TC 2	18	Minimum Age	Age Accepted
TC 3	19	Minimum Age +1	Age Accepted
TC 4	59	Maximum Age -1	Age Accepted
TC 5	60	Maximum Age	Age Accepted
TC 6	61	Maximum Age +1	Not Allowed/Invalid

B. Describe static analysis tools and explain its importance.

Answer

Static Analysis Tools

Static analysis tools are software tools used to examine the source code of a program without executing the program. These tools help identify programming errors, security vulnerabilities, coding standard violations, and logical mistakes during the development stage itself. Static analysis improves the quality and reliability of software before testing and deployment.

Examples of Static Analysis Tools

- SonarQube
- ESLint
- Checkstyle
- PMD
- Coverity

Importance of Static Analysis Tools

- **Early Detection of Errors:** Static analysis tools identify bugs and defects in the early stages of software development, reducing debugging efforts later.
- **Improves Code Quality:** These tools help developers write clean, readable, and maintainable code by checking coding standards and best practices.
- **Enhances Software Security:** They can detect security vulnerabilities such as buffer overflows, SQL injection risks, and unsafe coding practices.
- **Reduces Development Cost:** Fixing errors during coding is cheaper and easier than fixing them after software deployment.
- **Saves Time:** Automated analysis is faster than manual code review and helps developers quickly identify issues.
- **Improves Maintainability:** Well-structured and error-free code is easier to update, understand, and maintain in the future.

Example

If a programmer uses an uninitialized variable in a program, a static analysis tool can detect the issue before the software is executed.

C . For the given code, calculate the cyclomatic complexity.

Read A

If A>0 Then

If A=21 Then

Print "Key"

End If

End If

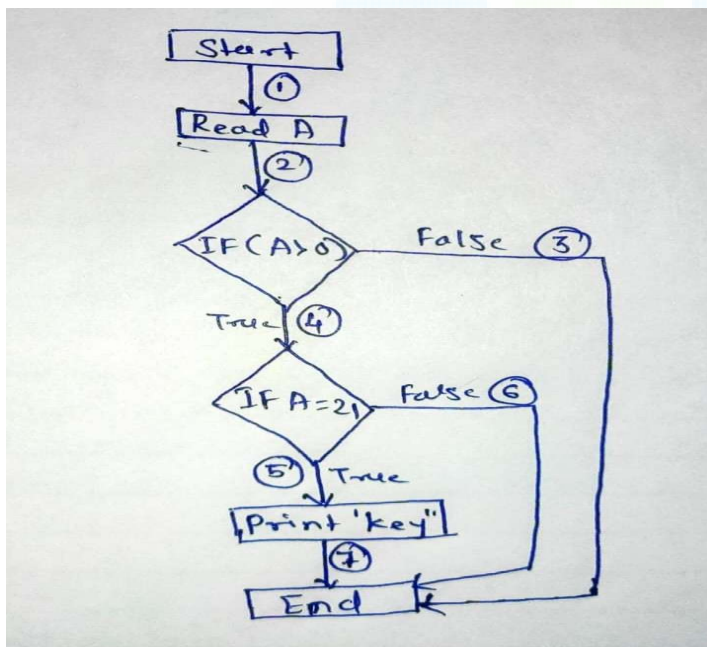
Answer

i) Find Nodes

1. Start
2. Read A
3. If A>0
4. If A=21
5. Print Key
6. End

Total Nodes : 6

ii) Find Edges



Total Edges =7

Formula for calculating Cyclomatic Complexity

$$V(G) = E - N + 2P$$

Where

E= Edges

N= Node

P= Piece of Code

$$= E - N + 2P$$

$$= 7 - 6 + 2 * 1$$

$$= 1 + 2$$

$$= 3$$

So, Cyclomatic Complexity = 3

Independent Path =3

Another method (for direct Calculation)

Total Decision (if / else if) + 1

In above program 2 if condition

Formula = D+ 1

$$= 2 + 1$$

$$= 3$$

D. Differentiate between black box and white box testing.

Aspect	White Box Testing	Black Box Testing
Meaning	White Box Testing is a type of testing where the internal code and logic of the program are tested.	Black Box Testing is a type of testing where the functionality of the software is tested without knowing the internal code.
Knowledge of Code	In White Box Testing, the tester must have knowledge of the code.	In Black Box Testing, the tester does not need knowledge of the code.
Focus	White Box Testing focuses on internal structure like logic, conditions, and paths.	Black Box Testing focuses on input and output behavior of the system.
Performed By	White Box Testing is usually performed by developers or skilled testers.	Black Box Testing is performed by testers or end users.
Test Design	In White Box Testing, test cases are designed based on the code structure.	In Black Box Testing, test cases are designed based on requirements.
Objective	The objective of White Box Testing is to find logical and coding errors.	The objective of Black Box Testing is to check whether the system works as expected.
Level of Testing	White Box Testing is mainly used in unit testing.	Black Box Testing is mainly used in system and acceptance testing.
Example	For example, checking all if-else conditions in a program.	For example, checking login functionality with valid and invalid inputs.

Question 4 (10 Marks Each)

A . An online learning platform, Knowledge, provides interactive courses, video lectures and quizzes. It supports user roles such as students, instructors and administrators. The platform includes features like personalized learning recommendations, discussion forum, progress tracking and integration with third-party tools like payment gateways.

Write a test plan for the following sections of the IEEE 829 test plan template for the above application:

- i) Scope of testing
- ii) Objectives
- iii) Risks
- iv) Strategy
- v) Approach

Answer

Test Plan for "Knowledge" Online Learning Platform (IEEE 829 Template)

i) Scope of Testing

What we WILL test:

- Student features – browse courses, watch video lectures, attempt quizzes, track progress.
- Instructor features – upload courses, create quizzes, reply to discussions.
- Admin features – manage users, approve courses, view reports.
- Personalized recommendations – system suggests relevant courses to students.
- Discussion forum – students can post questions, instructors can answer.
- Payment gateway integration – course purchase, subscription, refunds.
- Third-party tools – video streaming service, payment gateways.

What we will NOT test:

- Quality of video content (e.g., blurry video, poor audio).
- External payment gateway's internal security (only integration is tested).
- Student's internet connection or device hardware.

ii) Objectives

Objectives explain **why testing is done for the online learning platform**.

- **To ensure all features are working correctly:** All functions like login, course access, video lectures, quizzes, and payments should work without errors.
- **To provide a smooth user experience:** Students and instructors should be able to use the system easily without confusion, delays, or crashes.
- **To verify correctness of quizzes and progress tracking:** Quiz marks, results, and student progress reports should be accurate and updated properly.
- **To ensure secure payments and protect user data:** Payment transactions should be safe, and personal data (like login details) must be protected from unauthorized access.
- **To identify and fix bugs before release:** Testing helps find errors in the system so they can be corrected before users start using the platform.
- **To improve quality and reliability of the system:** The system should perform well under different conditions and give consistent results every time.

iii) Risks

- **Video Streaming Failure:** Third-party video server may go down, stopping all lectures. Students will not be able to watch courses, leading to bad user experience and loss of trust.
- **Payment Gateway Downtime:** If the payment gateway is down, students cannot purchase new courses during peak hours. This directly causes loss of revenue for the platform.
- **Quiz data Loss:** If a server crashes while a student is taking a quiz, all answers may be lost. Students will have to restart the quiz, causing frustration and wasted time.
- **Wrong Recommendations:** System may suggest irrelevant courses or courses already completed by the student. This makes the personalized learning feature useless and annoys users.
- **Discussion Forum Spam:** Without proper spam filtering, users may post abusive content, ads, or irrelevant messages. This ruins the learning environment and may drive away genuine students.
- **Progress Mismatch:** Student completes a course but progress still shows as incomplete. This demotivates learners and creates confusion about what is actually finished.

- **Role Access Violation:** A student may accidentally or intentionally access instructor features (like uploading courses). This can corrupt course data and create security issues.
- **Poor Performance during Peak Time:** When many students watch videos or take quizzes together, the platform may become slow or crash. This happens during exam time or new course launches.

iv) Strategy

- **Unit testing** – Test small functions like "calculate quiz score" and "update progress".
- **Integration testing** – Test connection between platform and payment gateway, video server.
- **System testing** – Test full flow: student login → buy course → watch video → take quiz → see progress.
- **UI testing** – Check screens on desktop, tablet, and mobile.
- **Performance testing** – Test with 1000 students watching videos simultaneously.
- **Security testing** – Check that one student cannot access another student's data.
- **Regression testing** – Re-test old features after every bug fix.

v) Approach (Step-by-step)

Step 1: Planning – Prepare test cases for login, course purchase, video play, quiz, forum, progress tracking.

Step 2: Environment setup – Setup test server, test payment gateway, mock video streaming service.

Step 3: Functional testing – Test positive flows (buy course → watch → take quiz → see progress). Test negative flows (payment fail → no access, wrong answer → low score).

Step 4: Role-based testing – Login as student (should only see courses). Login as instructor (should upload courses). Login as admin (should approve users).

Step 5: Recommendation testing – Complete 2 courses in "Python" → check if the system suggests more Python courses.

Step 6: Real-time progress testing – Watch half lecture → check progress shows 50%. Complete quiz → check marks added.

Step 7: Performance testing – Simulate 500 students watching videos at same time. Check buffering time and server load.

Step 8: Security testing – Try to access another student's course or progress – system must block.

Step 9: Bug tracking – Log bugs in Jira. Developer fixes → retest → close bug.

Step 10: Sign-off – All critical bugs fixed, 95% tests passed, performance and security okay.

B .Write Suitable test cases for above application

Answer

Test Case ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Status
TC_01	Student Login	Valid student account exists	Enter valid login details and click Login	Student dashboard should open	As per execution	Pass/Fail
TC_02	Invalid Login	Login page available	Enter invalid username/pass word	Error message should display	As per execution	Pass/Fail
TC_03	Course Purchase	Student logged in	Select course and complete payment	Course should be added successfully	As per execution	Pass/Fail
TC_04	Video Playback	Student enrolled in course	Open and play lecture video	Video should play properly	As per execution	Pass/Fail
TC_05	Quiz Submission	Quiz available	Attempt and submit quiz	Correct score should display	As per execution	Pass/Fail

TC_06	Progress Tracking	Student enrolled in course	Watch lecture partially	Progress should update correctly	As per execution	Pass/Fail
TC_07	Recommendation Feature	Student completed course	Open recommendation section	Related courses should display	As per execution	Pass/Fail
TC_08	Discussion Forum Post	User logged in	Post message in forum	Message should appear in forum	As per execution	Pass/Fail
TC_09	Instructor Upload Course	Instructor account available	Upload course material	Course upload should succeed	As per execution	Pass/Fail
TC_10	Admin User Management	Admin logged in	Block/unblock user	User status should update	As per execution	Pass/Fail
TC_11	Payment Failure	Payment page open	Enter invalid card details	Payment failure message should display	As per execution	Pass/Fail
TC_12	Unauthorized Access	Student logged in	Try opening admin page	Access should be denied	As per execution	Pass/Fail
TC_13	Password Reset	Registered email exists	Click forgot password	Reset link should be sent	As per execution	Pass/Fail
TC_14	Mobile Compatibility	Platform opened on mobile	Open dashboard and courses	Pages should display properly	As per execution	Pass/Fail
TC_15	Performance Testing	Multiple users active	Simulate many users	System should not crash	As per execution	Pass/Fail

Question 5 (5 Marks Each)

A. How can a testing tool be successfully introduced and integrated into an organization's workflow?

Answer

Successfully introducing and integrating a testing tool into an organization's workflow requires proper planning, training, and continuous improvement. The following steps help in successful integration:

Key Steps Involved

1. **Identify the Need :** First, the organization identifies a problem or requirement where the tool can help, such as improving testing, communication, or project management. This step ensures that the tool is actually needed and will solve a real problem. It helps avoid unnecessary investment.
2. **Tool Selection :** Different tools are evaluated based on cost, features, compatibility, and ease of use. The best-suited tool is selected. Comparison of multiple tools helps in making the right decision. Proper selection increases efficiency and reduces future issues.
3. **Planning and Preparation :** A proper plan is created for implementation, including budget, timeline, and resources required. Roles and responsibilities are also clearly defined. This ensures smooth execution without confusion.
4. **Installation and Setup:** The tool is installed and configured according to the organization's needs. System requirements and settings are properly adjusted. This step makes the tool ready for actual use.
5. **Training Employees :** Staff members are trained so they can use the tool effectively and confidently. Training sessions, demos, or manuals may be provided. It reduces errors and increases productivity.
6. **Pilot Testing :** The tool is tested on a small scale to check if it works properly before full implementation. Any issues or bugs are identified and fixed at this stage. It reduces risk before full deployment.
7. **Full Implementation:** After successful testing, the tool is deployed across the organization. All users start using the tool in a real work environment. This step marks complete adoption of the tool.
8. **Monitoring and Support :** Performance is monitored, and technical support is provided to

resolve issues. Regular updates and maintenance are also carried out. It ensures long-term smooth functioning of the tool.

B. Discuss the selenium components-WebDriver and TestNG

Answer

1. WebDriver

Selenium WebDriver

WebDriver is an important component of Selenium used for automating web application testing. It directly communicates with web browsers and performs actions like clicking buttons, entering text, opening webpages, and validating results. WebDriver supports different browsers such as Chrome, Firefox, and Edge. It helps testers execute automated test cases quickly and accurately. It is widely used because it is open-source and easy to integrate with testing frameworks.

Features:

- Supports multiple browsers
- Supports languages like Java and Python
- Faster execution
- Automates browser activities

Example:

```
WebDriver driver = new ChromeDriver();
```

```
driver.get("https://example.com");
```

Advantages:

- Reduces manual testing effort
- Improves testing speed
- Open-source and easy to use

2. TestNG

TestNG is a testing framework used with Selenium for managing and executing test cases. It provides features like annotations, test grouping, prioritization, and report generation. TestNG makes automated testing more organized and efficient. It helps testers run multiple test cases in sequence and generate detailed reports. TestNG also supports parallel execution, which saves testing time.

Features:

- Uses annotations like @Test
- Supports parallel testing
- Generates test reports
- Allows test prioritization

Example:

@Test

```
public void loginTest() {  
  
    System.out.println("Login Successful");  
  
}
```

Advantages:

- Better test management
- Easy integration with Selenium
- Automatic report generation

C. Explain the benefits of JIRA in software Test management.**Answer**

JIRA is a popular project management and bug tracking tool used in software testing. It helps testers, developers, and project managers manage testing activities, track defects, and monitor project progress efficiently. JIRA improves communication among team members and helps maintain proper records of bugs and test cases. It is widely used in Agile and Scrum development environments for effective software test management.

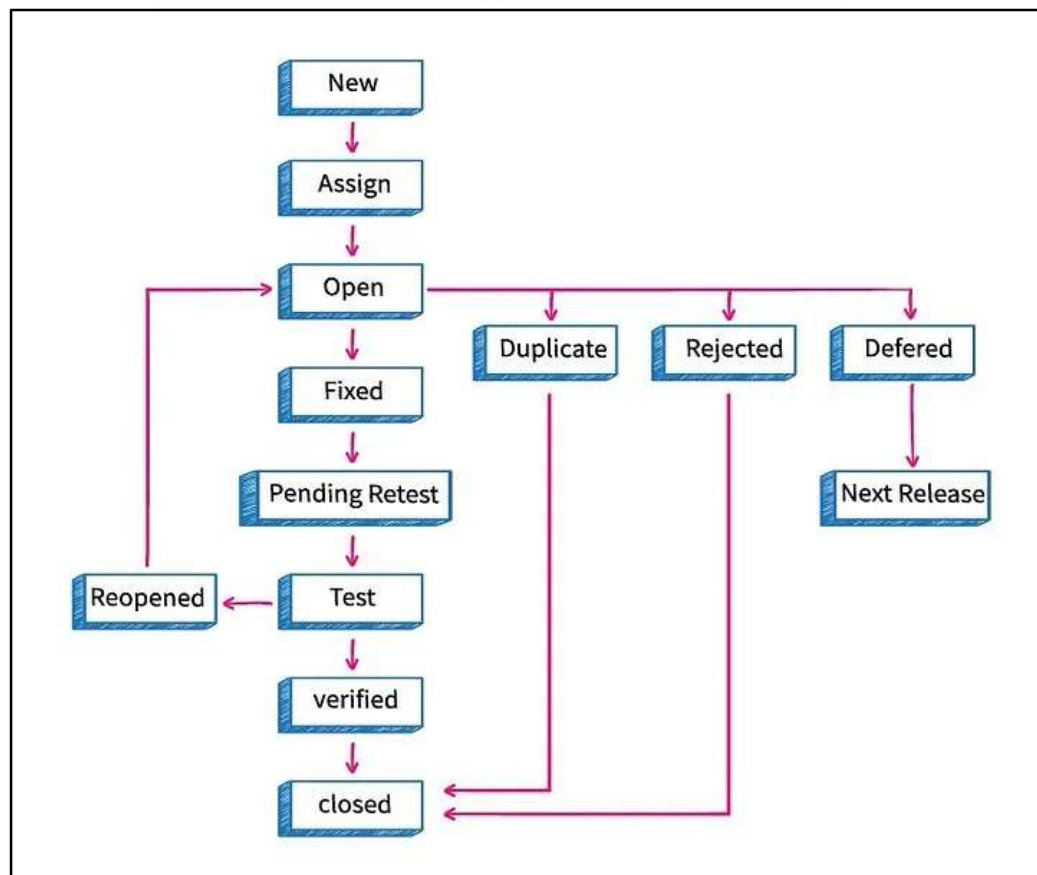
Benefits of JIRA in Software Test Management

1. **Bug Tracking:** JIRA helps testers report, track, and manage software defects easily. It allows teams to monitor bug status until the issue is resolved. This improves software quality and reduces errors.
2. **Better Team Collaboration:** Developers, testers, and managers can communicate through comments and updates in JIRA. This improves coordination among team members. It also helps in faster problem resolution.

3. **Test Case Management:** JIRA helps organize and manage test cases efficiently. Test execution progress and results can also be tracked properly. This makes testing activities more systematic.
4. **Real-Time Project Monitoring:** JIRA provides dashboards and reports to monitor testing progress, pending tasks, and defect status in real time. It helps managers make better decisions quickly.
5. **Integration with Testing Tools:** JIRA can integrate with tools like Selenium and TestNG. This helps automate testing and improves overall efficiency. It also saves testing time and effort.
6. **Supports Agile Methodology:** JIRA supports Agile practices like Scrum and Kanban. It helps teams manage sprints, tasks, and testing activities effectively. This improves project management and productivity.

D. Narrate the stages of the Defect Life cycle with a diagram.

Answer



1. **New** – A defect is identified and recorded for the first time. It is logged into the defect tracking system with all details.No action is taken yet at this stage.
2. **Assign** – The defect is assigned to a developer or team member. Responsibility for fixing the issue is clearly defined. The developer reviews the defect details before starting work.
3. **Open** –The developer begins working on the defect.The issue is analyzed to find the root cause.Necessary changes are planned to fix it.
4. **Fixed** –The developer corrects the defect in the code.Required changes are implemented and updated. The defect is now ready for testing.
5. **Test** –The tester checks whether the defect is fixed properly. Different test cases are executed to validate the fix. The tester ensures no new issues are introduced.
6. **Verified** – The tester confirms that the defect is resolved successfully. The fix works as expected in all scenarios.The defect is ready to be closed.
7. **Closed** – The defect is completely resolved and finalized. No further testing or changes are required. It is marked as completed in the system.
8. **Reopened** –If the defect still exists, it is opened again. The previous fix did not work correctly. It is sent back to the developer for correction.
9. **Duplicate** –The defect is already reported earlier. It matches an existing defect in the system. So, it is marked as duplicate to avoid repetition.
10. **Rejected** – The defect is not valid or cannot be reproduced. It may be due to misunderstanding or incorrect data. So, it is not considered for fixing.
11. **Deferred** –The defect is valid but not fixed immediately. It is postponed due to low priority or time constraints. It will be handled in future if needed.
12. **Next Release** – The defect is planned to be fixed in the next version.It is not critical for the current release. It will be resolved in upcoming updates.